

PeakClip

Komplettübersicht · Architektur · Prozesse · Neuerungen

YouTube → KI-Clips → Shorts / TikTok / Reels

Version 2.0 · Juni 2026

Produktion: Hetzner VPS · /opt/videclip

PeakClip — Komplettübersicht & Neuerungen

Version: 2.0 · Stand: Juni 2026

Produktion: Hetzner VPS 62.238.39.31 · Pfad /opt/videclip

1. Was ist PeakClip?

PeakClip (intern: Videclip) wandelt lange YouTube-Videos und Uploads in **virale Short-Form-Clips** um — optimiert für **YouTube Shorts, TikTok und Reels** (Standard **9:16**).

Schritt	Beschreibung
Import	YouTube-URL oder lokales Video hochladen
KI-Analyse	Claude findet die besten Highlight-Momente
Clip-Erstellung	Schnitt, Transkription, Rendering mit Untertiteln
Bearbeitung	Trim, Hook, Bild, Overlays, Musik, Untertitel
Export	Einzelclip oder ZIP mit allen Clips

2. System-Architektur (Übersicht)



▼▼▼

bgutil POT WARP SOCKS5 Puppeteer
(Port 4416) (Port 40000) (Cookie-Refresh)

Tech-Stack

Ebene	Technologie
Frontend	React 18, Vite, Tailwind CSS
Backend	Node.js (ESM), Express
KI Highlights	Anthropic Claude Sonnet
Sprache/STT	Groq Whisper large-v3
Video	FFmpeg, ffprobe, yt-dlp
Thumbnails	Python rembg + OpenCV, Node sharp
Hosting	Hetzner CX, Ubuntu, systemd

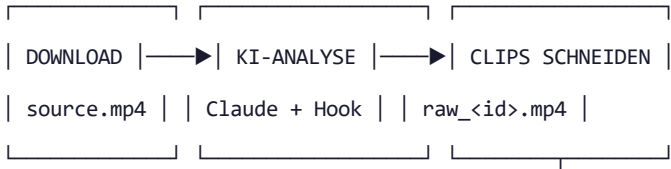
3. Analyze-Pipeline (Server)

Einstieg: POST /api/analyze → runAnalyzeJob()

Prozess-Skizze (vereinfacht)

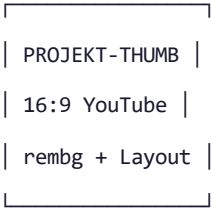
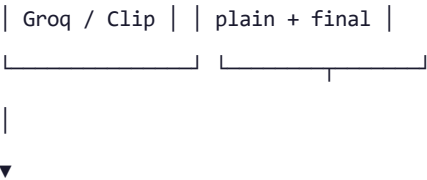
[URL / Upload]

▼



▼▼





Phasen im Detail

Phase	Aktion	Fortschritt (ca.)
0	Video laden, Metadaten, Claude-Highlight-Erkennung	10-30 %
1	Rohclips schneiden (parallel)	50 %
2	Alle Clips transkribieren (Groq)	65 %
3	Clips rendern (9:16, Untertitel, Plain-Overview)	85 %
4	Projekt-Thumbnail generieren (16:9)	90 %
5	meta.json, Projekt registrieren	100 %

UI-Fortschritt (4 Schritte – NEU)

Früher gab es 5 Schritte mit separatem „Untertitel“. Transkription läuft weiter im Hintergrund, in der UI sind es nur noch **4 Schritte**:

#	Schritt	Server-Step(s)
1	Import	fetching
2	KI-Analyse	analyzing
3	Clips erstellen	editing + transcribing
4	Fertig	ready

4. Editor & Clip-Bearbeitung (Frontend)

6-Schritte-Editor



Schritt 1 – Clip Ausschnitt wählen (Trim-Slider)

- Schritt 2 – Hook Cold-open, Spannungs-Moment
- Schritt 3 – Bild Aspect Ratio (9:16 / 1:1 / 16:9)
- Schritt 4 – Overlays Wide-Overlay, Webcam-PiP
- Schritt 5 – Musik Preset oder Upload
- Schritt 6 – Untertitel Style, CTA-Text



Aspect Ratio – Regeln (NEU)

Bereich	Format	Erklärung
Clip-Vorschau & Export	9:16 (Standard)	Shorts/TikTok/Reels
Projekt-Dashboard	16:9	YouTube-Style Thumbnails
Clip-Karten (Overview)	9:16	Vertikale Vorschau-Clips

Der Editor startet immer in 9:16, unabhängig von gespeicherten Einstellungen.

Live-Trim-Vorschau (NEU)

Beim Verschieben der Trim-Griffe:



[9:16 Video-Frame]

Quellvideo wird direkt im Browser

auf Start/Ende gesprungen –

KEIN Server-Render beim Trimmen

▶

0:12 / 0:38 LIVE



- Eigene Player-Leiste (Play, Scrubber, eine Zeitangabe)
- Keine doppelten Browser-Controls mehr
- Server-Render nur bei aktiven Effekten (Untertitel, Hook, Musik, ...)

Hook-Logik (NEU)



hook_peak_time (KI)

|

▼

Liegt Peak im Ausschnitt?

/ \

JA NEIN

| |

| ▼

| Clip automatisch um Hook

| neu ausrichten (buildTrimAroundHook)

| |

|_____|

▼

Cold-open aktivieren möglich

(Teaser + Cliffhanger-Text)

|

|

|

Hook-Quellen (Priorität):

|

- hook_peak_time aus KI-Analyse

|

- zoom_moments[0] als Fallback

|

- Berechneter Peak bei ~68 % der Original-Clip-Länge

Bei Cold-open AN und Hook außerhalb: Clip wird **automatisch** am Hook ausgerichtet.

5. Automatische Projekt-Thumbnails (NEU)

|

Nach jeder Analyse wird thumbs/project.jpg erzeugt – 16:9, YouTube-Style.

|

Pipeline

|

|

Bestes Frame (Gesicht + Schärfe)

|

└─▶ Gameplay-Hintergrund (links, blur)

|

└─▶ Streamer freistellen (rembg)

|



Weißer Stroke (PIL)

|



Compositing (sharp 1280×720)

+ Titel + Episodennummer + Mood-Farbe

|



thumbs/project.jpg



|

Datei	Funktion
server/scripts/thumbnail_prep.py	Frame-Wahl, Webcam-Ecke, rembg
server/src/services/projectThumbnail.js	Layout, Text, Export
server/requirements-thumbnail.txt	opencv, rembg, Pillow

Projekt-Karten zeigen dieses 16:9-Thumbnail. **Clip-Karten** bleiben 9:16.

6. Produktionsserver (Hetzner)

Infrastruktur

Dienst	Zweck
videclip.service	Haupt-App (Node)
bgutil-pot.service	PO-Token für yt-dlp (Port 4416)
warp-proxy.service	Cloudflare WARP SOCKS5 (Port 40000)

YouTube-Download-Stack



|

yt-dlp

└─ Cookies (cookies.txt)

└─ bgutil PO-Token (127.0.0.1:4416)

└─ WARP Proxy (socks5://127.0.0.1:40000)

└─ Player-Clients: mweb, web_safari, android_vr



Wichtige .env -Einträge (Server)

COOKIES_PATH=/opt/videclip/server/cookies.txt

COOKIES_SKIP_BROWSER_EXPORT=1

YTDLP_PATH=/usr/local/bin/yt-dlp

YTDLP_POT_BASE_URL=http://127.0.0.1:4416

YTDLP_PROXY=socks5://127.0.0.1:40000

PYTHON_PATH=python3

Wartung

Aufgabe	Aktion
Logs	journalctl -u videclip -f
Cookies abgelaufen	cookies.txt vom PC per SCP hochladen
Client deployen	npm run build + client/dist auf Server
Thumbnail-Deps	pip install -r requirements-thumbnail.txt

Wichtig: WARP nur als **Proxy-Modus**, nicht Full-Tunnel (SSH-Probleme).

7. Neuerungen – Zusammenfassung

Feature	Beschreibung	Status
4-Schritte-Pipeline	Untertitel in „Clips erstellen“ integriert	Live
Projekt 16:9 / Clip 9:16	Getrennte Aspect Ratios	Live
Auto-Projekt-Thumbnail	rembg + YouTube-Layout	Live
Live-Trim	Sofortige Vorschau ohne Server-Wait	Live
Clip-Player	Eigene Controls, eine Zeitangabe	Live
Hook auto-align	Clip richtet sich am Hook aus	Live
Hetzner Production	WARP + bgutil + Cookies	Live
Puppeteer headless	Kein X-Server auf VPS	Live

8. Ordnerstruktur (Job-Workspace)

```
%TEMP%/videoclip/<jobId>/
├─ source.mp4 Quellvideo
├─ meta.json Highlights, Settings, projectThumbnailUrl
├─ raw-clips/ Rohschnitte
├─ transcripts/<id>/ STT pro Clip
├─ clips/ Fertige Clips + plain_<id>.mp4 (9:16 Overview)
├─ thumbs/
│   └─ <id>.jpg Clip-Thumbnails
│   └─ project.jpg 16:9 Projekt-Thumbnail (NEU)
│   └─ project_prep/ rembg-Zwischendateien
└─ previews/ Editor-Previews nach Änderungen
```

9. API-Endpunkte (Kern)

Methode	Pfad	Funktion
POST	/api/analyze	Analyze-Job starten
GET	/api/job/:id	Job-Status pollen
POST	/api/preview	Editor-Vorschau rendern
POST	/api/process	Final-Export eines Clips
GET	/api/files/:jobId/...	Videos, Thumbs, Clips
GET	/api/projects	Projekt-Liste

10. Vorschau-Cache (Tiered Preview)

Beim Bearbeiten im Editor werden FFmpeg-Pässe gecacht:

- Tier raw → Trim/Hook geändert → Neuer Rohschnitt
- Tier pass1 → Aspect/Crop/Grade geändert
- Tier pass2 → Untertitel/Hook-Text geändert

Tier music → Nur Musik/Mix geändert (schnell)

Tier noop → Cache-Hit, sofortige Antwort

Nur Trim ohne Effekte = Live-Vorschau im Browser (kein Tier nötig).

11. Deployment vom Windows-PC

powershell

Client bauen

cd C:\Users\rapha\Desktop\videclip

npm run build

Auf Server kopieren

scp -i C:\Users\rapha\.ssh\id_ed25519_hetzner -r client\dist root@62.238.39.31:/opt/videclip/client/

Server-Dateien (bei Backend-Änderungen)

scp -i C:\Users\rapha\.ssh\id_ed25519_hetzner server\src\services*.js

root@62.238.39.31:/opt/videclip/server/src/services/

Service neu starten

ssh -i C:\Users\rapha\.ssh\id_ed25519_hetzner root@62.238.39.31 "systemctl restart videclip"

12. Glossar

Begriff	Bedeutung
Cold-open / Hook	Kurzer Teaser vor dem Hauptclip (Scroll-Stopper)
Plain Overview	9:16-Vorschau ohne Effekte nach Analyse
Live-Trim	Browser-Vorschau beim Trimmen ohne FFmpeg
rembg	KI-Hintergrundentfernung für Thumbnails
PO-Token	YouTube-Bypass-Token via bgutil
Mood	hype / chill / emotional — beeinflusst KI-Auswahl

